Post-contenido — Instrucciones y direccionamiento

Arquitectura de Computadores · Unidad 6 · Post6-U6 Estudiante: González López **Repositorio:** TrabajosArqui — post6-u6 **PDF:** Gonzalez Lopez_Post6_U6.pdf

Alcance y entorno

Este laboratorio contiene dos programas NASM para DOSBox: `src/lab6_instrucciones.asm` y `src/lab6_modos.asm`. `src/lab6_factorial.asm` implementa la variante del factorial de 5. Los ejecutables `.COM` se guardan en `bin/`.

Los programas `.COM` son de 16 bits y usan `BITS 16`, `ORG 100h` y un segmento compartido en modo real. Por ello, se usan AX/BX/CX/DX, no EAX/EBX ni secciones ELF `.data/.text`. El ejemplo teórico de RIP-relative y el byte SIB pertenece a x86-64/32-bit protegido, no se puede ensamblar como direccionamiento escalado dentro de este `.COM`: en 16 bits, el recorrido indexado válido usa combinaciones como `[BX+SI]`, sin escala. En un `.COM`, DS y SS suelen compartir el mismo segmento; el ejemplo BP sigue mostrando la convención arquitectónica SS por defecto.

Requisitos: DOSBox 0.74 o superior, NASM para DOS disponible dentro del montaje y DEBUG.COM/DEBUG.EXE. El NASM de Windows puede usarse para compilar desde el host, pero no sustituye la comprobación de `nasm -v` dentro de DOSBox solicitada por la actividad. DEBUG no viene necesariamente incluido en DOSBox.

En `dosbox.conf`, actualizar la ruta de montaje si el repositorio está en otra ubicación. Los comandos de ensamblado desde el prompt DOS son:

```
nasm -f bin src\lab6_instrucciones.asm -o bin\lab6_instrucciones.com
nasm -f bin src\lab6_factorial.asm -o bin\lab6_factorial.com
nasm -f bin src\lab6_modos.asm -o bin\lab6_modos.com
```

Después de ensamblar, ejecutar cada `.COM` desde DOSBox. Los mensajes de fin confirman que el programa llegó a la terminación DOS; los valores intermedios deben inspeccionarse con DEBUG, no inferirse de una captura inexistente. Los binarios distribuidos se ensamblaron en Windows con NASM 2.16.03 para ejecutarse en DOSBox 0.74-3. La ejecución automática de DOSBox en esta sesión no produjo una señal de finalización verificable; por tanto, se marcan pendientes las capturas y la comprobación con DEBUG.

Parte 1 — Categorías de instrucciones

`lab6_instrucciones.asm` guarda los resultados en variables con nombre para facilitar la inspección. `flags_add`, `flags_sub`, `flags_and`, `flags_test` y `flags_cmp` conservan FLAGS (el registro de 16 bits en este programa) inmediatamente después de cada operación, mediante PUSHF/POP, antes de que otra instrucción altere los flags.

Bloque	Instrucciones y propósito	Resultado esperado
Transferencia	MOV, LEA, XCHG, PUSH/POP	AX vuelve a 45; SI recibe el offset de `valor_a`.
Aritmética	ADD, SUB, INC, DEC, MUL, DIV	suma 57; resta -33 (`FFDFh`); producto 70; división 100/7: cociente 14 y resto 2.
Lógica	AND, OR, XOR, TEST, SHL, SHR	`B7h AND 0Fh = 07h`; `B7h OR F0h = F7h`; `AAh XOR FFh = 55h`; TEST no cambia AL.
Flujo	CMP/JG/JE, LOOP y DEC/JNZ	`valor_a > valor_b`; ambas sumas de 1 a 5 dan AX=15.
Rotación	SHL/ROL/SHR/ROR y ROL por 4	con AL=`81h`: SHL=`02h`, ROL=`03h`, SHR=`40h`, ROR=`C0h`; `A5h` rotado 4 bits queda `5Ah`.

En ADD `2Dh+0Ch=39h`: CF=0, ZF=0, SF=0, OF=0. En SUB `0Ch-2Dh=FFDFh`: CF=1, ZF=0, SF=1, OF=0. AND y TEST sobre los valores anteriores producen resultado no nulo; CF=0 y OF=0. CMP `2Dh-0Ch=21h`: CF=0, ZF=0, SF=0, OF=0, por lo que JG toma la rama mayor. `MUL` define CF/OF según si la mitad alta del producto es distinta de cero; sus otros flags no están definidos. DIV no define flags. Se debe distinguir esta semántica documentada de los valores concretos que muestre DEBUG.

`lab6_factorial.asm` es la variante independiente del bucle: inicia AX=1 y multiplica por BX=1…5 con `MUL`, guardando 120 en `resultado`. La comparación LOOP y DEC/JNZ en el programa principal calcula la misma suma, para comparar claridad y posibles diferencias de rendimiento sin atribuir ciclos medidos.

Parte 2 — Modos de direccionamiento

Modo	Fórmula de dirección efectiva	Instrucción demostrada	Valor esperado
Inmediato	No se lee una dirección de datos: el valor forma parte de la instrucción	`mov ax,100`	AX=0064h
Directo	DS:desplazamiento codificado por el símbolo	`mov ax,[var_x]`	AX=FFFFh antes de poner `var_x` en cero
Indirecto por registro	DS:SI	`mov ax,[si]` después de `mov si,nota1`	AX=0055h (85)
Indexado	DS:(BX+SI) módulo 16 bits	`mov ax,[bx+si]`, con BX=`array`, SI=4	AX=001Eh (30), `array[2]`
Índice de array	Base + índice en bytes	`add ax,[bx+si]` en un bucle	suma ascendente e inversa: AX=0096h (150)
BP / locales	SS:(BP+desplazamiento)	`mov ax,[bp-2]`, `add ax,[bp-4]`	locales 99 y 250; suma 015Dh (349)
Matriz	Base + fila×ancho-fila×tamaño + columna×tamaño	`mov dx,[bx+si]`	SI=000Ah; DX=0006h para matriz[1][2]
Matriz directa	DS:(offset matriz + 10)	`mov ax,[matriz+10]`	AX=0006h para el elemento fijo matriz[1][2]

Trazado indirecto esperado

Los desplazamientos son offsets del programa y la pila inicial varía entre sesiones. Para el acceso indirecto, DEBUG debe mostrar SI apuntando a `nota1`; después de `MOV AX,[SI]`, AX=0055h. Valores resultantes esperados, no transcripción de una sesión de DEBUG:

Instrucción	Registro modificado	Valor esperado
`mov si,nota1`	SI	offset de `nota1`
`mov ax,[si]`	AX	0055h
`mov si,nota2`	SI	offset de `nota2`
`mov bx,[si]`	BX	0049h
`add ax,bx`	AX	009Eh
`shr ax,1`	AX	004Fh; promedio truncado 79
`mov [si],ax` después de apuntar a promedio	Memoria	004Fh en `promedio`

Para comprobar la matriz, la fila 1 ocupa `1×3×2=6` bytes; la columna 2 añade `2×2=4` bytes. Por tanto, SI=10 y la palabra en `matriz+10` es 6. El modo directo de desplazamiento fijo lee la misma palabra sin calcular los índices en tiempo de ejecución. La matriz inversa suma los mismos cinco valores empezando con SI=8 y restando 2 por iteración.

Decisiones técnicas

1. SHL/SHR frente a ROL/ROR

Para escalar enteros sin signo por potencias de dos, se eligen SHL/SHR cuando el programa acepta el desbordamiento o el truncamiento propio del ancho del registro y necesita que los bits que salen no regresen por el extremo opuesto. SHL desplaza hacia la izquierda y deja el último bit expulsado en CF; SHR hace lo correspondiente hacia la derecha. Si se necesita conservar la secuencia circular de bits —por ejemplo, intercambiar nibbles, rotar un campo o construir ciertos checksums— conviene ROL/ROR, que reinserta el bit expulsado en el extremo contrario. Para AL=81h, SHL AL,1 produce 02h y CF=1: el bit 7 original ya no forma parte del byte. ROL AL,1 produce 03h y CF=1: ese bit queda preservado ahora en el bit 0. Aplicar SHL varias veces no puede recuperar la información descartada; OR solo puede activar bits, no reconstruir el nibble perdido. El código debe elegir la instrucción según la semántica requerida, no por una equivalencia matemática simplificada: el desplazamiento puede alterar flags y los resultados dependen del ancho y el desbordamiento. La decisión debe confirmar también si se requiere signo; para enteros con signo, SAR conserva el signo al desplazar a la derecha, a diferencia de SHR.

2. LOOP frente a DEC/JNZ

LOOP expresa compactamente “decrementar CX y repetir mientras no sea cero”, por lo que en un ejemplo de cinco iteraciones hace evidente la intención y mantiene el contador junto con el salto en una sola instrucción de fuente. Sin embargo, una sola instrucción visible no garantiza menos ciclos que DEC/JNZ: la latencia, el rendimiento de emisión, la predicción de rama y el soporte de cada

microarquitectura son distintos; en varios procesadores modernos LOOP puede ser más lento que una secuencia simple de decremento y salto. Para cinco iteraciones, cualquier diferencia absoluta es despreciable frente al costo de iniciar el programa y no altera la elección pedagógica. En un bucle ejecutado millones de veces, el costo por iteración sí puede acumularse, pero no se debe adivinar cuál versión gana: se compila, se mide en la CPU objetivo y se controla el costo de medición. DEC modifica flags, LOOP no los modifica; si el programa necesita conservar el resultado de una comparación entre iteraciones, esta diferencia también afecta la corrección. La decisión de este laboratorio prioriza LOOP por legibilidad y usa DEC/JNZ como contraste funcional, no afirma una cifra de ciclos observada.

3. Modo directo frente a indexado para una posición fija

Si la posición matriz[1][2] es constante desde la compilación, el acceso directo `[matriz+10]` permite que NASM codifique el desplazamiento del elemento y evita ejecutar instrucciones que calculen fila y columna. La variante indexada del ejercicio multiplica la fila por el ancho, convierte unidades a bytes, desplaza la columna y suma ambos offsets antes de leer el dato. Esas operaciones tienen un costo real en instrucciones, dependencias y flags; en cambio, el modo directo no paga ese cálculo dinámico. La comparación debe formularse con precisión: la dirección efectiva y la lectura de memoria siguen teniendo su costo en ambos casos, por lo que no se afirma que el modo directo elimine toda latencia ni que el formato directo siempre sea más veloz en cada CPU. El direccionamiento indexado se vuelve necesario y mantenible cuando los índices provienen de datos de ejecución, como las variables de un bucle que visita filas y columnas. Para sumar las nueve celdas, escribir nueve símbolos y accesos directos duplicaría código y sería propenso a omisiones; un bucle indexado sirve para matrices de cualquier dimensión compatible. El programa guarda ambos resultados y el estudiante puede contrastarlos en DEBUG: las dos rutas deben producir 6 para este elemento fijo.

4. BP frente a BX/SI y segmento por defecto

En el direccionamiento de 16 bits, BP se usa convencionalmente como ancla del marco de pila: parámetros y variables locales se expresan mediante offsets estables, aunque SP cambie durante PUSH, POP o reservas locales. Por regla arquitectónica, una dirección que incluye BP usa SS por defecto; en cambio, BX y SI normalmente referencian datos a través de DS. No son registros físicamente incompatibles, pero la convención y el segmento implícito dan significado distinto a sus usos. En un programa COM, DOS establece un segmento compartido para código, datos y pila, así que SS y DS coinciden y `[bp-2]` parece acceder al mismo segmento que `[bx+si]`. En un EXE con SS distinto de DS, ignorar esta regla puede leer o sobrescribir un offset homónimo de la pila en vez del dato; la instrucción puede ensamblar y fallar silenciosamente en la lógica. Si el diseño realmente requiere que una dirección basada en BP se interprete en DS, se fuerza un prefijo de sobrescritura, por ejemplo `mov ax,[ds:bp+2]`, y se documenta por qué no es un acceso normal a parámetros. En código mantenible se reserva BP para el marco y se usa BX/SI para arrays de DS, evitando overrides salvo que la organización de memoria lo justifique.

Guía de DEBUG y evidencia

En DOSBox, verificar `nasm -v`, compilar los tres fuentes y ejecutar cada `.COM`. Para la inspección, usar `DEBUG bin\lab6_instrucciones.com` o `DEBUG bin\lab6_modos.com`; `R` muestra registros y flags, `T` traza una instrucción, `U 100` desensambla y `D DS:100` muestra memoria. El binario no conserva nombres de

símbolos, así que se debe consultar la lista de NASM (`nasm -f bin -l archivo.lst ...`) para ubicar las variables y los bloques antes de formar los comandos de dump.

Las 11 capturas indicadas por la guía no están presentes; la lista de `capturas/README.md` identifica las evidencias pendientes. No se declaran como realizadas ni se sustituyen por valores inventados. Tampoco se realizaron commits: el requisito de seis commits descriptivos debe completarse en el repositorio del estudiante conforme avance el trabajo.

Referencias técnicas

- AMD. (2024). *AMD64 Architecture Programmer's Manual, Volume 1: Application Programming*. https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/programmer-references/24592.pdf
- Agner Fog. (2025). *Instruction tables: Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD and VIA CPUs*. https://www.agner.org/optimize/instruction_tables.pdf
- Intel. (2024). *Intel 64 and IA-32 architectures software developer's manual*. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
- NASM. (s. f.). *The Netwide Assembler documentation*. https://www.nasm.us/doc/